Polymarket

Security Review

Cantina Managed review by:
Giovanni Di Siena, Lead Security Researcher
Red-Swan, Security Researcher

March 27, 2026

Contents

1 Introduction **2**
1.1 About Cantina . . . 2
1.2 Disclaimer . . . 2
1.3 Risk assessment . . . 2
1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
2.1 Scope . . . 3

3 Findings **5**
3.1 Medium Risk . . . 5
3.1.1 Complementary settlement lacks taker-side aggregate reconciliation . . . 5
3.1.2 Zero amount fill can finalize zero-sized orders without economic execution . . . 5
3.1.3 Non-complementary matching lacks same-condition token pair validation . . . 6
3.1.4 Fee cap bypass on zero fill `BUY` orders allows a malicious operator to drain taker collateral . . . 8
3.1.5 Orders can freely specify collateral `tokenId=0`, resulting in collateral-for-collateral settlement and extraction . . . 9
3.2 Low Risk . . . 10
3.2.1 ERC-1155 Compliance . . . 10
3.2.2 Pre-approved orders can revert on malformed signatures due to validation order . . . 11
3.2.3 Complementary taker buy fee cap is enforced on declared fill rather than executed cash 11
3.2.4 `OrdersMatched` event `takerAmountFilled` is misreported for complementary taker buy matches . . . 11
3.2.5 Use of non-descriptive Solady role aliases . . . 14
3.2.6 `CollateralToken` wrap/unwrap calls from `CtfCollateralAdapter` bypass paused-asset validation . . . 14
3.3 Informational . . . 15
3.3.1 Code Style . . . 15
3.3.2 Invalid field elements `x2 >= P` are not explicitly rejected by `CTHelpers::getCollectionId` . . . 15
3.3.3 The point at infinity is not explicitly rejected by `CTHelpers::getCollectionId` . . . 16
3.3.4 Non-canonical parent collection encodings can cause collection id collisions . . . 17
3.3.5 `CtfCollateralAdapter` passthrough partition allows non-binary splits and merges 18
3.3.6 Unused `OrderExpired` custom error . . . 18
3.3.7 Strict inequality in zero-taker case rejects valid boundary-crossing matches . . . 18
3.3.8 Zero-maker sell order is treated as a zero-price ask and falsely crosses any buy . . . 19
3.3.9 Users should be paused within `blockPausedAt` . . . 19
3.3.10 Repeated `pauseUser()` executions will delay pausing . . . 19
3.3.11 User accounts can be unpaused before or after the effective pause block has passed 20
3.3.12 Incorrect `takerFillAmount` documentation in `CTFExchange::matchOrders` . . . 20
3.3.13 Invalidated pre-approved orders can be reapproved without restriction . . . 20
3.3.14 Redundant non-zero fee validation before `_chargeFee()` calls . . . 20
3.3.15 Missing ci foundry profile leaves ffi enabled in `pull_request` tests . . . 21
3.3.16 Auth mappings should use boolean as flags instead of `uint256` . . . 21
3.3.17 Missing NatSpec on public/external functions . . . 21
3.3.18 Internal/Private Functions Missing Leading Underscores . . . 21

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Feb 24th to Mar 10th the Cantina team conducted a review of ctf-exchange-v2 on commit hash dafcfc0c. The team identified a total of **31** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	5	5	0
Low Risk	6	6	0
Gas Optimizations	0	0	0
Informational	20	13	7
Total	**31**	**24**	**7**

2.1 Scope

The security review had the following components in scope for ctf-exchange-v2 on commit hash dafcfc0c:

```
src
├── adapters
│   ├── CtfCollateralAdapter.sol
│   ├── NegRiskCtfCollateralAdapter.sol
│   └── libraries
│       └── CTHelpers.sol
├── collateral
│   ├── CollateralOfframp.sol
│   ├── CollateralOnramp.sol
│   ├── CollateralToken.sol
│   ├── PermissionedRamp.sol
│   ├── abstract
│   │   ├── CollateralErrors.sol
│   │   └── Pausable.sol
│   └── interfaces
│       ├── ICollateralToken.sol
│       └── ICollateralTokenCallbacks.sol
└── exchange
    ├── CTFExchange.sol
    ├── interfaces
    │   ├── IAssetOperations.sol
    │   ├── IAssets.sol
    │   ├── IAuth.sol
    │   ├── IConditionalTokens.sol
    │   ├── IFees.sol
    │   ├── IHashing.sol
    │   ├── IPausable.sol
    │   ├── ISignatures.sol
    │   ├── ITrading.sol
    │   └── IUserPausable.sol
    ├── libraries
    │   ├── CalculatorHelper.sol
    │   ├── Create2Lib.sol
    │   ├── PolyProxyLib.sol
    │   ├── PolySafeLib.sol
    │   ├── Structs.sol
    │   └── TransferHelper.sol
    └── mixins
        ├── AssetOperations.sol
```

```
        ├── Assets.sol
        ├── Auth.sol
        ├── ERC1155TokenReceiver.sol
        ├── Events.sol
        ├── Fees.sol
        ├── Hashing.sol
        ├── Pausable.sol
        ├── PolyFactoryHelper.sol
        ├── Signatures.sol
        ├── Trading.sol
        └── UserPausable.sol
```

3 Findings

3.1 Medium Risk

3.1.1 Complementary settlement lacks taker-side aggregate reconciliation

Severity: Medium Risk

Context: Trading.sol#L85-L93, Trading.sol#L148-L157

Description: The complementary branch of `_matchOrders()` computes the taker-side taking amount within the call to `_performOrderChecks()` and then returns early into `settleComplementary()` without enforcing any minimum. The non-complementary path, by comparison, does enforce this via `_updateTakingWithSurplus()`.

In complementary settlement, `totalTakerReceives` is simply accumulated from maker `fillAmount` and there is no aggregate check against the expected `taking` output amount. There is also no aggregate check that total maker-derived taker debits stay within `takerFillAmount`. As such, operator-controlled `makerFillAmounts` can make complementary execution order consumption diverge from the taker's intent.

If the actual taker making amount is greater than `takerFillAmount`, the taker can be overcharged while `orderStatus.remaining` is not sufficiently decremented, enabling repeated over-withdrawal against the same signed order. Conversely, if the actual taker making amount is less than `takerFillAmount`, the order can be over-consumed in storage without corresponding economic execution.

Recommendation: Consider passing the minimum `taking` amount into `_settleComplementary()` to enforce that the taker receives at least the expected taking amount (`totalTakerReceives >= taking`) and validate that aggregate complementary debits are bounded by the specified `takerFillAmount`.

Polymarket: Fixed in PR 59.

Cantina Managed: Verified. Complementary settlement will now revert if the executed taker making amount (sum of maker implied taker making amounts) exceeds the specified `takerFillAmount`. Similarly, the executed taker taking amount (sum of maker fill amounts) is enforced to be at least the minimum taker taking amount computed on the aggregate from the taker's own ratio.

3.1.2 Zero amount fill can finalize zero-sized orders without economic execution

Severity: Medium Risk

Context: Trading.sol#L578-L592

Description: When an order is first seen, `_updateOrderStatus()` initializes `remaining` to `order.makerAmount`; however, if an order has `makerAmount == 0` and the operator passes `makingAmount == 0`, the `makingAmount <= remaining` passes, causing `remaining` to stay as 0 and the packed status writes `filled = true` into storage.

This allows a zero-sized order to be marked as filled (and associated fill events emitted) even though no assets moved. In mixed batches, this can consume/invalidate such orders and create misleading execution history without real settlement.

Proof of Concept: The following test should be added to `MatchOrders.t.sol`:

```
function test_MatchOrders_PoC_MixedBatch_ZeroFillMakerMarkedFilled() public {
    uint256 davePK = 0xD00D;
    address dave = vm.addr(davePK);

    // Taker funds BUY YES; second maker funds BUY NO to force non-all-complementary
    ↪  path.
    dealUsdcAndApprove(bob, address(exchange), 50_000_000);
    dealUsdcAndApprove(dave, address(exchange), 50_000_000);
    // Give complementary maker approval so the zero-amount transfer path is executable.
    dealOutcomeTokensAndApprove(carla, address(exchange), yes, 1);

    // Taker: BUY YES
    Order memory takerOrder = _createAndSignOrder(bobPK, yes, 50_000_000, 100_000_000,
    ↪  Side.BUY);
```

```
    // Maker 0: COMPLEMENTARY SELL with zero-sized order.
    Order memory zeroComplementaryMaker = _createAndSignOrder(carlaPK, yes, 0, 0,
    ↪  Side.SELL);
    // Maker 1: MINT-side order so the batch is not all complementary.
    Order memory mintMaker = _createAndSignOrder(davePK, no, 50_000_000, 100_000_000,
    ↪  Side.BUY);

    bytes32 zeroComplementaryHash = exchange.hashOrder(zeroComplementaryMaker);
    assertFalse(exchange.getOrderStatus(zeroComplementaryHash).filled);
    assertEq(exchange.getOrderStatus(zeroComplementaryHash).remaining, 0);

    Order[] memory makerOrders = new Order[](2);
    makerOrders[0] = zeroComplementaryMaker;
    makerOrders[1] = mintMaker;

    uint256[] memory fillAmounts = new uint256[](2);
    fillAmounts[0] = 0;
    fillAmounts[1] = 50_000_000;

    uint256[] memory makerFeeAmounts = new uint256[](2);
    makerFeeAmounts[0] = 0;
    makerFeeAmounts[1] = 0;

    vm.prank(admin);
    exchange.matchOrders(conditionId, takerOrder, makerOrders, 50_000_000, fillAmounts,
    ↪  0, makerFeeAmounts);

    // Zero-fill complementary maker order is marked filled.
    assertTrue(exchange.getOrderStatus(zeroComplementaryHash).filled);
    assertEq(exchange.getOrderStatus(zeroComplementaryHash).remaining, 0);

    // No assets moved for the zero-fill maker order.
    assertCollateralBalance(carla, 0);
    assertCTFBalance(carla, yes, 1);

    // The mixed batch still executes through the MINT path.
    assertCollateralBalance(bob, 0);
    assertCTFBalance(bob, yes, 100_000_000);
    assertCollateralBalance(dave, 0);
    assertCTFBalance(dave, no, 100_000_000);
}
```

Recommendation: Consider rejecting zero-sized maker orders, and either reject `makingAmount == 0` in matching paths or treat it as a no-op that does not mutate order status or emit fill events.

Polymarket: Fixed in PR 69.

Cantina Managed: Verified. All orders are now explicitly prevented from being zero-sided.

3.1.3 Non-complementary matching lacks same-condition token pair validation

Severity: Medium Risk

Context: Trading.sol#L534

Description: Complementary matches enforce that orders on both sides reference the same `tokenId`. Given that `conditionId` is a single-market context for the batch, there should also be similar validation for non-complementary (`BUY+BUY/SELL+SELL`) orders; however, it is not currently enforced that both `tokenIds` correspond to the expected outcome pair (`YES/NO`) of the `conditionId`.

This validation gap allows multiple bad shapes in non-complementary matching, including:

1. Cross-condition mismatch (for example `YES_A` matched with `YES_B`, or `NO_B` in mixed batches, which can execute when stale inventory exists to satisfy payout).
2. Mixed, same-condition batches with same-side matching (for example a maker `BUY YES_A` when the taker is also `BUY YES_A`).

This is a valid concern for both match types, but it is more immediate for `MINT`: `splitPosition()` creates both outcomes, while settlement pays makers the requested `tokenId` and sweeps the taker surplus from the taker-side balance, so mixed batches can drain one side and leave residual tokens on the other. By contrast, `MERGE` needs equal balances of both outcomes for the full amount, and mixed `SELL` `tokenIds` generally fail this condition and revert.

Without the validation, stale inventory can satisfy maker payouts for unrelated condition IDs, so an invalid pair can execute instead of reverting. Given that `MINT` batches only enforce price crossing, a mixed batch can also create stale inventory from clean state, with the opposite side outcome remaining on the exchange. This residual balance is fully backed but unowned, so a subsequent taker can claim it via surplus given that `_updateTakingWithSurplus()` returns the full contract balance for `takerAssetId`.

Proof of Concept: The following test should be added to `MatchOrders.t.sol`:

```
function
↪  test_MatchOrders_BadNonComplementaryPair_DifferentConditions_WithExchangeInventory()
↪  public {
    // Deals: both traders are BUYers, so they only need collateral.
    dealUsdcAndApprove(bob, address(exchange), 50_000_000);
    dealUsdcAndApprove(carla, address(exchange), 50_000_000);

    // Prepare a second binary condition and use its NO token id.
    bytes32 otherConditionId = _prepareCondition(admin, bytes32(uint256(0x5678)));
    uint256 otherNo = ctf.getPositionId(address(usdc), ctf.getCollectionId(bytes32(0),
    ↪  otherConditionId, 1));

    // Create stale otherNo inventory via a mixed same-condition MINT batch
    ↪  (otherConditionId):
    // taker BUY otherYes, maker1 BUY otherNo, maker2 BUY otherYes.
    {
        uint256 otherYes = ctf.getPositionId(address(usdc),
        ↪  ctf.getCollectionId(bytes32(0), otherConditionId, 2));
        (address seedTaker, uint256 seedTakerPK) = makeAddrAndKey("seed-taker");
        (address seedMakerNo, uint256 seedMakerNoPK) = makeAddrAndKey("seed-maker-no");
        (address seedMakerYes, uint256 seedMakerYesPK) =
        ↪  makeAddrAndKey("seed-maker-yes");

        dealUsdcAndApprove(seedTaker, address(exchange), 100_000_000);
        dealUsdcAndApprove(seedMakerNo, address(exchange), 50_000_000);
        dealUsdcAndApprove(seedMakerYes, address(exchange), 50_000_000);

        Order[] memory seedMakerOrders = new Order[](2);
        seedMakerOrders[0] = _createAndSignOrder(seedMakerNoPK, otherNo, 50_000_000,
        ↪  100_000_000, Side.BUY);
        seedMakerOrders[1] = _createAndSignOrder(seedMakerYesPK, otherYes, 50_000_000,
        ↪  100_000_000, Side.BUY);

        uint256[] memory seedFillAmounts = new uint256[](2);
        seedFillAmounts[0] = 50_000_000;
        seedFillAmounts[1] = 50_000_000;

        uint256[] memory seedMakerFeeAmounts = new uint256[](2);

        Order memory seedTakerOrder = _createAndSignOrder(seedTakerPK, otherYes,
        ↪  100_000_000, 100_000_000, Side.BUY);

        vm.prank(admin);
        exchange.matchOrders(
            otherConditionId,
            seedTakerOrder,
            seedMakerOrders,
            100_000_000,
            seedFillAmounts,
            0,
            seedMakerFeeAmounts
        );
```

```
    }

    assertCTFBalance(address(exchange), otherNo, 100_000_000);

    // Invalid non-complementary pairing: BUY YES (condition A) vs BUY NO (condition B).
    // If exchange has otherNo inventory, this can execute instead of reverting.
    Order memory yesBuyTaker = _createAndSignOrder(bobPK, yes, 50_000_000, 100_000_000,
    ↪  Side.BUY);
    Order memory noBuyOtherCondition = _createAndSignOrder(carlaPK, otherNo, 50_000_000,
    ↪  100_000_000, Side.BUY);

    Order[] memory makerOrders = new Order[](1);
    makerOrders[0] = noBuyOtherCondition;

    uint256[] memory fillAmounts = new uint256[](1);
    fillAmounts[0] = 50_000_000;

    uint256[] memory makerFeeAmounts = new uint256[](1);
    makerFeeAmounts[0] = 0;

    vm.prank(admin);
    exchange.matchOrders(conditionId, yesBuyTaker, makerOrders, 50_000_000, fillAmounts,
    ↪  0, makerFeeAmounts);

    // Taker receives YES from condition A, maker receives NO from condition B.
    assertCTFBalance(bob, yes, 100_000_000);
    assertCTFBalance(carla, otherNo, 100_000_000);

    // Exchange consumed stale otherNo inventory and now holds leftover NO from condition
    ↪  A.
    assertCTFBalance(address(exchange), otherNo, 0);
    assertCTFBalance(address(exchange), no, 100_000_000);
}
```

Recommendation: Derive the two valid outcome token IDs for the provided `conditionId` and require `takerOrder.tokenId` to be one of those two IDs. For each maker, non-complementary matches should require `makerOrder.tokenId` to be the opposite ID in that same YES/NO pair.

Polymarket: Fixed in commits 38aac63 and fd2f1a9.

Cantina Managed: Verified. The supplied `tokenIds` are now verified to be valid positions in the set for the given `conditionId` across all orders, preventing orders from being matched across different markets. Token IDs are now also validated to differ for `MINT/MERGE` types which, with the above validation, prevents invalid non-complementary matches. Furthermore, maker exchange orders are now settled with delta-based surplus to prevent pre-existing balance from inflating the actual taking amount.

3.1.4 Fee cap bypass on zero fill `BUY` orders allows a malicious operator to drain taker collateral

Severity: Medium Risk

Context: Fees.sol#L49, Trading.sol#L148, Trading.sol#L463-L467

Description: `CTFExchange::matchOrders` does not enforce a minimum number of maker orders. When called with empty maker arrays and `takerFillAmount = 0`, `Trading::_performOrderChecks()` computes a `cashValue` of zero for taker `BUY` fee validation. However, `Fees::validateFeeWithMaxFeeRate()` skips enforcement when `cashValue == 0` while the complementary branch of `Trading::_matchOrders` proceeds in charging `takerFeeAmount` given that `_isAllComplementary()` defaults to true for an empty array. This allows an operator to charge arbitrary taker fees without executing any trade, leaving the order effectively unfilled and reusable. Therefore, by matching a taker `BUY` order with empty maker orders, a malicious operator can steal the full principal through fees.

Proof of Concept: The following test should be added to `MatchOrders.t.sol`:

```
function test_MatchOrders_PoC_OperatorStealsFromTakerBuy_WithNoMakers() public {
    uint256 takerFillAmount = 50_000_000;
```

```
    uint256 takerFeeAmount = 2_500_000; // 5% of takerFillAmount (max allowed by default)
    uint256 totalExpectedSpend = takerFillAmount + takerFeeAmount;

    // Bob funds/approves the full expected spend for a correct match (principal + fee).
    dealUsdcAndApprove(bob, address(exchange), totalExpectedSpend);

    Order memory takerOrder = _createAndSignOrder(bobPK, yes, takerFillAmount,
    ↪  100_000_000, Side.BUY);
    bytes32 takerOrderHash = exchange.hashOrder(takerOrder);

    // Empty maker arrays
    Order[] memory makerOrders = new Order[](0);
    uint256[] memory fillAmounts = new uint256[](0);
    uint256[] memory makerFeeAmounts = new uint256[](0);

    // Operator matches with no makers but still charges taker fee.
    // By passing a zero `takerFillAmount`, Bob's entire approved balance can be stolen
    ↪  through fees.
    // This happens because the cash value is zero, so fee validation is skipped.
    vm.prank(admin);
    exchange.matchOrders(
        conditionId, takerOrder, makerOrders, 0, fillAmounts, totalExpectedSpend,
        ↪  makerFeeAmounts
    );

    // Order not marked as filled and remaining is still the full `takerFillAmount` so
    ↪  future approvals can also be repeatedly stolen.
    assertFalse(exchange.getOrderStatus(takerOrderHash).filled);
    assertEq(exchange.getOrderStatus(takerOrderHash).remaining, takerFillAmount);

    // Bob loses all principal + fee collateral, receives no outcome tokens.
    assertCollateralBalance(bob, 0);
    assertCollateralBalance(feeReceiver, totalExpectedSpend);
    assertCTFBalance(bob, yes, 0);
    assertCollateralBalance(address(exchange), 0);
    assertCTFBalance(address(exchange), yes, 0);
    assertCTFBalance(address(exchange), no, 0);
}
```

Recommendation: Consider explicitly rejecting empty maker batches and require non-zero taker fill in matching.

Polymarket: Fixed in commit 5936812.

Cantina Managed: Verified. The fee cap is now enforced on zero-value fills.

3.1.5 Orders can freely specify collateral `tokenId=0`, resulting in collateral-for-collateral settlement and extraction

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: Asset id 0 is a special case that represents ERC-20 collateral transfers, but signed orders are never enforced to have `tokenId != 0`. This breaks the core invariant that `Order.tokenId` must represent a CTF ERC-1155 transfer, meaning that the intended bidirectional collateral-CTF exchange can be made to collapse into collateral-collateral flows. Given that the non-complementary branch does not enforce correct `tokenId` semantics, when matched against a normal victim order with a valid non-zero `tokenId`, this can be used by an attacker to break the core accounting and consume surplus collateral paid out as if it were an outcome token. Complementary matches require `takerOrder.tokenId == makerOrder.tokenId`, so in this case the victim would have to erroneously specify `tokenId=0`.

Proof of Concept: The following test should be added to `MatchOrders.t.sol`:

```
function test_MatchOrders_PoC_TokenIdZero_Mint_ExtractsCollateral() public {
    // Honest taker places a normal YES BUY; attacker places BUY with tokenId=0.
    dealUsdcAndApprove(bob, address(exchange), 150_000_000);
```

```
    dealUsdcAndApprove(carla, address(exchange), 50_000_000);

    Order memory takerYesBuy = _createAndSignOrder(bobPK, yes, 150_000_000, 100_000_000,
    ↪  Side.BUY);
    // 1) Attacker is a BUY maker with tokenId=0.
    // For BUY, _deriveAssetIds() computes (makerAssetId=0, takerAssetId=tokenId), so
    ↪  this becomes (0, 0).
    Order memory attackerCollateralBuy = _createAndSignOrder(carlaPK, 0, 50_000_000,
    ↪  100_000_000, Side.BUY);

    Order[] memory makerOrders = new Order[](1);
    makerOrders[0] = attackerCollateralBuy;

    uint256[] memory fillAmounts = new uint256[](1);
    fillAmounts[0] = 50_000_000;

    uint256[] memory makerFeeAmounts = new uint256[](1);
    makerFeeAmounts[0] = 0;

    // 2) Maker settlement pays out via takerAssetId:
    // _distributeMakerProceeds() -> _transfer(exchange -> maker, takerAssetId,
    ↪  takingAmount).
    // Since takerAssetId == 0 here, the attacker is paid collateral, not outcome tokens.
    vm.prank(admin);
    exchange.matchOrders(conditionId, takerYesBuy, makerOrders, 150_000_000,
    ↪  fillAmounts, 0, makerFeeAmounts);

    // 3) Collateral source:
    // Combined deposits are taker 150 + attacker 50. After MINT consumes 100 collateral,
    ↪  the protocol still
    // pays the attacker 100 collateral, leaving zero collateral in the exchange.
    // Attacker doubles collateral despite not receiving any outcome tokens.
    assertCollateralBalance(carla, 100_000_000);
    assertCTFBalance(carla, yes, 0);
    assertCTFBalance(carla, no, 0);

    // 4) Result:
    // Taker gets YES but no collateral refund (refund path checks _getBalance(0), which
    ↪  is now 0),
    // so they lose the full collateral budget.
    assertCollateralBalance(bob, 0);
    assertCTFBalance(bob, yes, 100_000_000);
}
```

Recommendation: In addition to validating token semantics for non-complementary matches, consider enforcing that the specified `tokenId` cannot be zero within the order validation.

Polymarket: Fixed in PR 60.

Cantina Managed: Verified. The `tokenId=0` case is now implicitly prevented by the `_validateTokenIds()` logic which computes and validates a

3.2 Low Risk

3.2.1 ERC-1155 Compliance

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: The `ERC1155TokenReceiver` contract implements `ERC-1155`'s `ERC1155TokenReceiver` interface but does not implement `ERC-165`'s `supportsInterface` method as the ERC demands.

Recommendation: Consider implementing `ERC-165` in the `ERC1155TokenReceiver` contract.

Polymarket: Fixed in PR 70.

Cantina Managed: Verified. `supportsInterface()` has been added to `ERC1155TokenReceiver`, returning true for the `ERC1155TokenReceiver` and `ERC-165` interface IDs.

3.2.2 Pre-approved orders can revert on malformed signatures due to validation order

Severity: Low Risk

Context: Signatures.sol#L45-L51

Description: Pre-approved orders can fail validation in `validateOrderSignature()` if signature data is malformed. This is because `isValidSignature()` is evaluated first and will revert in `ECDSA.recover()`. Additionally, the function name is somewhat misleading considering the `OR` requirement.

Recommendation: Consider renaming this function and reversing the order of conditions such that validation will pass for pre-approved orders even if the signature data is malformed.

Polymarket: Fixed in PR 74.

Cantina Managed: Verified. 0 byte signatures are subject to pre-approval validation, whereas orders with malformed signatures are now explicitly rejected.

3.2.3 Complementary taker buy fee cap is enforced on declared fill rather than executed cash

Severity: Low Risk

Context: Trading.sol#L230-L231, Trading.sol#L463-L467

Description: Fee rate validation performed in `_performOrderChecks()` is based on the specified `takerFillAmount` rather than the actual taker cash spent in complementary execution (sum of maker-side taking values) which could be smaller. However, the taker `BUY` conditional in `_settleComplementaryTaker()` still charges the full `takerFeeAmount`, meaning that the fee cap is enforced against input parameters but not against actual execution which could cause issues if the matching engine/operator is buggy/malicious.

Consider the following scenario in which the max fee rate is assumed to be 5%:

1. Operator submits a taker `BUY` with `takerFillAmount = 100 USDC` and `takerFeeAmount = 5 USDC` (passes cap: 5% of 100).
2. Complementary maker fills execute only 20 USDC of actual taker cash spend.
3. Protocol still charges the full 5 USDC taker fee in `_settleComplementaryTaker()`.
4. Effective fee on executed cash is `5 / 20 = 25%`, exceeding the intended cap.

Recommendation: Consider computing the actual taker `BUY` cash spend (aggregate taker collateral debits to makers, excluding taker fee) in complementary settlement and revalidate against the fee cap before charging.

Polymarket: Fixed in PR 59.

Cantina Managed: Verified. The taker fee is now validated against the actual executed taker making amount derived from order consumption implied by maker fills.

3.2.4 `OrdersMatched` event `takerAmountFilled` is misreported for complementary taker buy matches

Severity: Low Risk

Context: Trading.sol#L253

Description: In the complementary settlement path, the `totalTakerReceives` accumulator emitted as the `takerAmountFilled` field of the `OrdersMatched` event is only populated for taker `SELL` matches. For taker `BUY` matches, the ternary operator falls back to `takerFillAmount` which is collateral spent in maker-side units, not outcome tokens received. This causes the event to report `takerAmountFilled` in the wrong unit for complementary taker buys when the price is not 1:1.

Recommendation: Consider refactoring the per-maker complementary return value from a sell-specific collateral implementation to a unified taker-side received amount, tracking this value for both taker `BUY`

and taker `SELL` paths. This allows a single `totalTakerReceives` to be accumulated in taker-side units and unconditionally emitted across all complementary makers.

Consider also validating that the aggregate outcome token amount received by the taker for a complementary `BUY` order is at least the minimum required by the taker order's price for the given `takerFillAmount` does not consume `takerFillAmount` in order but deliver less than calculated minimum. If the matched makers deliver less than that minimum, revert.

```
  diff --git a/src/exchange/mixins/Trading.sol b/src/exchange/mixins/Trading.sol
  index c51e4f5..7497f7b 100644
-  -- a/src/exchange/mixins/Trading.sol
+  ++ b/src/exchange/mixins/Trading.sol
  @@ -87,7 +87,13 @@ abstract contract Trading is Hashing, AssetOperations, Events, Fees,
  ↪  UserPausabl
           // Check if all matches are COMPLEMENTARY (all makers have opposite side to
           ↪  taker)
           if (_isAllComplementary(takerOrder.side, makerOrders)) {
               _settleComplementary(
-                  takerOrder, makerOrders, makerFillAmounts, makerFeeAmounts, orderHash,
↪  takerFillAmount, takerFeeAmount
+                  takerOrder,
+                  makerOrders,
+                  makerFillAmounts,
+                  makerFeeAmounts,
+                  orderHash,
+                  takerFillAmount,
+                  takerFeeAmount
               );
               return;
           }
  @@ -140,28 +146,41 @@ abstract contract Trading is Hashing, AssetOperations, Events,
  ↪  Fees, UserPausabl
           uint256 takerFillAmount,
           uint256 takerFeeAmount
       ) internal {
-          address taker = takerOrder.maker;
-          bool takerIsBuy = takerOrder.side == Side.BUY;
           uint256 totalMakerFees = 0;
           uint256 totalTakerReceives = 0;
+          uint256 totalTakerAmountFilled = 0;

           for (uint256 i = 0; i < makerOrders.length;) {
               (uint256 makerFee, uint256 makerCollateral) = _settleComplementaryMaker(
-                  takerOrder, makerOrders[i], makerFillAmounts[i], makerFeeAmounts[i],
↪  taker, takerIsBuy
+                  takerOrder, makerOrders[i], makerFillAmounts[i], makerFeeAmounts[i]
               );
               unchecked {
                   totalMakerFees += makerFee;
                   totalTakerReceives += makerCollateral;
+                  totalTakerAmountFilled += makerFillAmounts[i];
                   ++i;
               }
           }

+          require(
+              totalTakerAmountFilled
+                  >= CalculatorHelper.calculateTakingAmount(takerFillAmount,
↪  takerOrder.makerAmount, takerOrder.takerAmount),
+              TooLittleTokensReceived()
+          );
+
           // Batch transfer accumulated maker SELL fees from taker
-          if (totalMakerFees > 0) _transfer(taker, getFeeReceiver(), 0, totalMakerFees);
+          if (totalMakerFees > 0) _transfer(takerOrder.maker, getFeeReceiver(), 0,
↪  totalMakerFees);
```

```diff
          // Handle taker settlement
          _settleComplementaryTaker(
-             takerOrder, taker, takerIsBuy, totalTakerReceives, takerOrderHash,
↪  takerFillAmount, takerFeeAmount
+             takerOrder,
+             takerOrder.maker,
+             takerOrder.side == Side.BUY,
+             totalTakerReceives,
+             totalTakerAmountFilled,
+             takerOrderHash,
+             takerFillAmount,
+             takerFeeAmount
          );
      }

@@ -172,10 +191,11 @@ abstract contract Trading is Hashing, AssetOperations, Events,
↪  Fees, UserPausabl
          Order memory takerOrder,
          Order memory makerOrder,
          uint256 fillAmount,
-         uint256 feeAmount,
-         address taker,
-         bool takerIsBuy
+         uint256 feeAmount
      ) internal returns (uint256 makerFee, uint256 makerCollateral) {
+         address taker = takerOrder.maker;
+         bool takerIsBuy = takerOrder.side == Side.BUY;
+
          _validateOrdersMatch(takerOrder, makerOrder, MatchType.COMPLEMENTARY);

          (uint256 taking, bytes32 orderHash) = _performOrderChecks(makerOrder,
          ↪  fillAmount, feeAmount);
@@ -223,6 +243,7 @@ abstract contract Trading is Hashing, AssetOperations, Events,
↪  Fees, UserPausabl
          address taker,
          bool takerIsBuy,
          uint256 totalTakerReceives,
+         uint256 takerAmountFilled,
          bytes32 takerOrderHash,
          uint256 takerFillAmount,
          uint256 takerFeeAmount
@@ -250,7 +271,7 @@ abstract contract Trading is Hashing, AssetOperations, Events,
↪  Fees, UserPausabl
                  side: takerOrder.side,
                  tokenId: takerOrder.tokenId,
                  makerAmountFilled: takerFillAmount,
-                 takerAmountFilled: takerIsBuy ? takerFillAmount : totalTakerReceives,
+                 takerAmountFilled: takerAmountFilled,
                  fee: takerFeeAmount,
                  builder: takerOrder.builder,
                  metadata: takerOrder.metadata
diff --git a/src/test/MatchOrders.t.sol b/src/test/MatchOrders.t.sol
index 79a5792..2b65562 100644
-  -- a/src/test/MatchOrders.t.sol
+  ++ b/src/test/MatchOrders.t.sol
@@ -774,11 +774,11 @@ contract MatchOrdersTest is BaseExchangeTest {

          vm.expectEmit(true, true, true, true);
          emit OrderFilled(
-             takerHash, bob, address(exchange), Side.BUY, yes, 50_000_000, 50_000_000,
↪  2_500_000, bytes32(0), bytes32(0)
+             takerHash, bob, address(exchange), Side.BUY, yes, 50_000_000, 100_000_000,
↪  2_500_000, bytes32(0), bytes32(0)
          );
```

```
            vm.expectEmit(true, true, true, true);
-           emit OrdersMatched(takerHash, bob, Side.BUY, yes, 50_000_000, 50_000_000);
+           emit OrdersMatched(takerHash, bob, Side.BUY, yes, 50_000_000, 100_000_000);

            vm.prank(admin);
            exchange.matchOrders(conditionId, takerOrder, makerOrders, 50_000_000,
            ↪  fillAmounts, 2_500_000, makerFeeAmounts);
```

Polymarket: Fixed in PR 62.

Cantina Managed: Verified. The event now emits `takerAmountFilled` in the correct unit corresponding to the actual amount of outcome-tokens received.

3.2.5 Use of non-descriptive Solady role aliases

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: `CollateralToken` defines aliases for the raw Solady bitmask roles:

```
uint256 internal constant MINTER_ROLE = _ROLE_0;
uint256 internal constant WRAPPER_ROLE = _ROLE_1;
```

However, the other contracts in the codebase use `_ROLE_0` and `_ROLE_1` directly. Each contract reuses these numeric slot names to represent different semantic roles. For example:

- `_ROLE_0` means `MINTER_ROLE` in `CollateralToken`, `ADMIN_ROLE` in the ramps, and an implicit admin role in `Pausable`.
- `_ROLE_1` means `WRAPPER_ROLE` in `CollateralToken` and `WITNESS_ROLE` in `PermissionedRamp`.

This creates two related problems.

- Readability: A reader auditing `PermissionedRamp` who also knows `_ROLE_0 == MINTER_ROLE` from CollateralToken may incorrectly conclude that ramp admins hold minting authority over the collateral token. Role storage is per-contract. `_ROLE_0` in `PermissionedRamp` has no relation to `_ROLE_0` in `CollateralToken` but the shared numeric name obscures this.
- Maintainability within inheritance chains: Role bit assignments must be non-overlapping across a contract's entire inheritance hierarchy. `CollateralOnramp`, `CollateralOfframp`, and `PermissionedRamp` all inherit from `Pausable`, which uses `_ROLE_0` for pause/unpause. Currently this is consistent. The inheriting contracts also treat `_ROLE_0` as admin but this is a matter of convention, not enforcement. A future developer adding a new role to a contract in this hierarchy, or extending `Pausable` to use additional bits, has no compiler-visible record of which bits are already assigned. A collision would silently grant unintended permissions to the wrong role holder.

Recommendation: Each contract that inherits `OwnableRoles` should define aliases for every role bit it uses and use these aliases exclusively. Where a contract inherits from a base that already defines role constants (e.g. Pausable), the named constants should be defined in the base and inherited rather than re-declared. This makes the full set of assigned bits visible in one place per inheritance tree, enabling a developer adding a new role to trivially verify there is no collision.

Polymarket: Fixed in PR 78.

Cantina Managed: Verified. Descriptive roles are now used.

3.2.6 `CollateralToken` wrap/unwrap calls from `CtfCollateralAdapter` bypass paused-asset validation

Severity: Low Risk

Context: CtfCollateralAdapter.sol#L47, CtfCollateralAdapter.sol#L73, CtfCollateralAdapter.sol#L90, CollateralOfframp.sol#L47, CollateralOnramp.sol#L47, PermissionedRamp.sol#L64-L71, PermissionedRamp.sol#L86-L93

Description: While it is understood that not all `CollateralOnramp`, `CollateralOfframp`, and `PermissionedRamp` contracts will be simultaneously deployed in the short term,

asset pause validation is currently enforced only in ramp contracts and not in `CollateralToken::wrap`/`CollateralToken::unwrap`. Because `CtfCollateralAdapter` calls `CollateralToken` directly, it can execute wrap/unwrap operations without passing ramp validation, allowing paused-asset protections to be bypassed.

Proof of Concept: Assume that USDCe is paused on the `CollateralOnramp` and `CollateralOfframp` contracts but USDC paths remain available and vault inventory exists.

Paused USDCe redemption is still reachable by calling `CtfCollateralAdapter::splitPosition`. The adapter unwraps `CollateralToken` to USDCe internally via `CollateralToken::unwrap` without a ramp pause check. The resulting YES/NO positions can then be merged directly via `ConditionalTokens::mergePositions` to recover USDCe, meaning a USDCe outflow happened despite ramp pause.

Similarly, minting using paused USDCe is still reachable by calling `CtfCollateralAdapter::mergePositions` or `CollateralToken::redeemPositions` for positions collateralized by USDCe. The adapter wraps USDCe into `CollateralToken` via `CollateralToken::wrap`, once again without a ramp pause check. Unwrap can then be performed into USDC, meaning a conversion from paused USDCe into active collateral happened despite ramp pause.

Recommendation: Consider modifying `CtfCollateralAdapter` to call the ramps directly for wrap/unwrap operations to ensure that asset pause state is respected during permissionless split/merge/redeem calls.

Polymarket: Fixed in PR 81.

Cantina Managed: Verified. `CtfCollateralAdapter` entrypoints can now only be called if USDCe is not paused. Note that this pause state must be separately maintained from the ramp contracts.

3.3 Informational

3.3.1 Code Style

Severity: Informational

Context: Hashing.sol#L11-L12

Description: There are several code locations that do not cohere to its own style norms. For example:

- There are several internal/private functions that do not have names with leading underscores even though this is the norm in this codebase. This includes almost all the functions in the `Signatures` contract. The `validateFeeWithMaxFeeRate` function in the `Fees` contract.
- In `Auth` there are several functions with parameter names that do not have trailing underscores like all other parameters in the codebase.
- In `Hashing` the constants `domainName` and `domainVersion` do not use upper case. Neither does `parentCollectionId` in `AssetOperations`.

Recommendation: Consider editing the code examples above to have a consistent style.

Polymarket: Fixed in PR 75.

Cantina Managed: Verfified.

3.3.2 Invalid field elements `x2 >= P` are not explicitly rejected by `CTHelpers::getCollectionId`

Severity: Informational

Context: CTFHelpers.sol#L17-L20, CTHelpers.sol#L424, CTHelpers.sol#L426-L427

Description: If `parentCollectionId` is ever user-supplied (rather than being hardcoded to `bytes32(0)` as it is currently) then this implementation is reliant on `ECADD` to reject invalid field elements `x2 >= P` that pass the earlier on-curve check. It may instead be preferable to revert early with an explicit error in this scenario.

Recommendation: Consider reverting with an explicit error when the `parentCollectionId` and hence `x2` is not a valid field element `(mod P)`.

Polymarket: Acknowledged. This is legacy code and `parentCollectionId` hardcoded to `bytes32(0)`.

Cantina Managed: Acknowledged.

3.3.3 The point at infinity is not explicitly rejected by `CTHelpers::getCollectionId`

Severity: Informational

Context: CTHelpers.sol#L417-L418, CTHelpers.sol#L428

Description: `CTHelpers::getCollectionId` treats `bytes32(0)` as a special sentinel case in which there is no parent collection. However, when `parentCollectionId != 0`, this function performs elliptic-curve addition and does not explicitly reject the point-at-infinity result `(0,0)`. If addition returns infinity, the compression step returns `bytes32(0)` which collides with the sentinel collection identifier.

Assuming the `parentCollectionId` can be freely specified, which is not currently the case', this scenario can be constructed by:

1. Deriving a base collection point `P` from `(parentCollectionId=0, conditionId, indexSet)`.
2. Flipping the encoded parity bit to create `-P` as a non-zero `parentCollectionId`.
3. Calling `getCollectionId(-P, conditionId, indexSet)`, which computes `P + (-P) = (0,0)`.

This causes a collision in which a non-zero-parent path produces the same `bytes32(0)` collection ID as the no-parent sentinel, breaking the invariant that collection IDs should be unique..

Proof of Concept: The following test demonstrates that a non-zero parent can cause `getCollectionId()` to erroneously return `bytes32(0)` by triggering point-at-infinity addition:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.30;

import { TestHelper } from "src/test/dev/TestHelper.sol";
import { Deployer } from "src/test/dev/util/Deployer.sol";
import { IConditionalTokens } from "src/adapters/interfaces/IConditionalTokens.sol";

contract CTHelpersPointAtInfinityTest is TestHelper {
    IConditionalTokens internal conditionalTokens;

    function setUp() public {
        conditionalTokens = IConditionalTokens(Deployer.deployConditionalTokens());
    }

    function test_CTHelpers_PoC_NonZeroParentCanProducePointAtInfinity() public {
        address oracle = alice;
        bytes32 questionId = keccak256("cthelpers.infinity.poc");

        conditionalTokens.prepareCondition(oracle, questionId, 2);
        bytes32 conditionId = conditionalTokens.getConditionId(oracle, questionId, 2);

        uint256 indexSet = 1;
        // Step 1: derive a base collection from the root (parent = 0). Internally this
        ↪  corresponds
        // to some curve point P encoded in the collection-id compression format.
        bytes32 baseCollectionId = conditionalTokens.getCollectionId(bytes32(0),
        ↪  conditionId, indexSet);
        assertTrue(baseCollectionId != bytes32(0), "unexpected zero base collection");

        // Step 2: create a NON-ZERO parent that decodes to -P (the additive inverse of
        ↪  P).
        // CTHelpers compresses points as:
        // - bits [0..253]: x
        // - bit 254: parity of y
        // Flipping bit 254 keeps x unchanged but swaps y <-> (p - y), i.e. P -> -P.
        bytes32 inverseParentCollectionId = bytes32(uint256(baseCollectionId) ^
        ↪  (uint256(1) << 254));
```

```
        assertTrue(inverseParentCollectionId != bytes32(0), "unexpected zero inverse
        ↪  parent");

        // Step 3: call getCollectionId with parent = -P and child = P.
        // The internal ecadd computes P + (-P) = point at infinity, represented by the
        ↪  precompile
        // return value (0,0). The implementation does not explicitly reject this result.
        //
        // Because y == 0, no parity bit is set, and x == 0 is returned directly as
        ↪  bytes32(0).
        // This collides with the sentinel encoding for "no parent collection", even
        ↪  though the
        // supplied parent here is non-zero.
        bytes32 combinedCollectionId =
        ↪  conditionalTokens.getCollectionId(inverseParentCollectionId, conditionId,
        ↪  indexSet);
        assertEq(combinedCollectionId, bytes32(0), "expected point at infinity
        ↪  encoding");
    }
}
```

Recommendation: Consider explicitly rejecting `ECADD` operations that result in the point-at-infinity. An additional defensive measure could be to enforce the invariant that when `parentCollectionId != 0`, `getCollectionId()` must never return `bytes32(0)`.

Polymarket: Acknowledged. This is legacy code and `parentCollectionId` hardcoded to `bytes32(0)`.

Cantina Managed: Acknowledged.

3.3.4 Non-canonical parent collection encodings can cause collection id collisions

Severity: Informational

Context: CTHelpers.sol#L419-L420

Description: `CTHelpers::getCollectionId` intends to use a canonical encoding for the `parentCollectionId` in which bit 254 stores y-parity and bit 255 is unused; however, the current implementation accepts non-canonical encodings since the parity is actually derived from both the highest bits - if either one is set then the point will be treated as odd. This means that distinct, non-canonical inputs with bit 255 set are still accepted and can decode to the same compressed EC point as canonical inputs, resulting in `parentCollectionId` collisions. In particular, top-bit patterns `01`, `10`, and `11` can all be interpreted as odd with the same x coordinate.

Proof of Concept: The following test demonstrates how a non-canonical `parentCollectionId` (with bit 255 additionally set) can collide to produce an identical child collection ID:

```
// SPDX-License-Identifier: MIT
pragma solidity <0.9.0;

import { Test } from "lib/forge-std/src/Test.sol";

import { CTHelpers } from "src/adapters/libraries/CTHelpers.sol";

contract CTHelpersTest is Test {
    function testFuzz_NonCanonicalParentX2Collides(bytes32 parentConditionId)
        public
        view
    {
        bytes32 conditionId = keccak256("conditionId");
        bytes32 canonicalParent = CTHelpers.getCollectionId(bytes32(0),
        ↪  parentConditionId, 1);

        // Force odd=1 and then set the non-canonical high bit: both decode to the same
        ↪  EC point.
        canonicalParent = bytes32(uint256(canonicalParent) | (1 << 254));
        bytes32 nonCanonicalParent = bytes32(uint256(canonicalParent) | (1 << 255));
```

```
        bytes32 child1 = CTHelpers.getCollectionId(canonicalParent, conditionId, 1);
        bytes32 child2 = CTHelpers.getCollectionId(nonCanonicalParent, conditionId, 1);

        // top-bit patterns 01, 10, 11 all decode as odd=true with same x.
        assertEq(child1, child2, "expected collision");
    }
}
```

Recommendation: Consider enforcing canonical parent encoding before decoding by explicitly rejecting inputs for which bit 255 is set. Alternatively, parity decoding could be performed from bit 254 only by explicitly masking out bit 255.

Polymarket: Acknowledged. This is legacy code and `parentCollectionId` hardcoded to `bytes32(0)`.

Cantina Managed: Acknowledged.

3.3.5 `CtfCollateralAdapter` passthrough partition allows non-binary splits and merges

Severity: Informational

Context: CtfCollateralAdapter.sol#L49, CtfCollateralAdapter.sol#L70

Description: `CtfCollateralAdapter::splitPosition` and `CtfCollateralAdapter::mergePositions` currently pass through the caller-supplied `_partition`; however, since the intention is for use with binary conditional tokens, `CTFHelpers.partition()` could be called directly here to enforce binary outcomes and make direct adapter calls behave equivalently to exchange calls.

Recommendation: Consider removing the `_partition` argument and replace it with a call to `CTFHelpers.partition()` within the internal functions.

Polymarket: Fixed in PR 76.

Cantina Managed: Verified. `_splitPosition()` and `_mergePositions()` now enforce the binary partition instead of accepting caller-supplied partitions.

3.3.6 Unused `OrderExpired` custom error

Severity: Informational

Context: ITrading.sol#L8

Description: The `OrderExpired` error is never used as there is no order expiry implemented.

Recommendation: Consider removing the unused error if order expiry logic is not needed.

Polymarket: Fixed in PR 70.

Cantina Managed: Verified. The unused error has been removed.

3.3.7 Strict inequality in zero-taker case rejects valid boundary-crossing matches

Severity: Informational

Context: CalculatorHelper.sol#L42-L43

Description: While it is not currently used within `Trading.sol`, `CalculatorHelper::isCrossing` intends to enforce that both sides must sum to no more than one to split; however, in the case where the `takerAmount` of one side is zero then the other is enforced to be strictly less than one.

If one side's `takerAmount` is zero, that side is effectively not contributing a meaningful bound, similar to an unspecified slippage constraint. Even in this circumstance, requiring the other side to be strictly less than one rejects the boundary case in which matches can be perfectly filled, blocking valid executions.

Recommendation: Consider modifying the conditions to use `<=`.

Polymarket: Fixed in PR 68.

Cantina Managed: Verified. The dead code has been removed.

3.3.8 Zero-maker sell order is treated as a zero-price ask and falsely crosses any buy

Severity: Informational

Context: CalculatorHelper.sol#L24-L35, CalculatorHelper.sol#L37-L39, CalculatorHelper.sol#L46-L47, CalculatorHelper.sol#L50-L61

Description: While `CalculatorHelper::isCrossing` is not currently used within `Trading.sol`, the internal call to `CalculatorHelper::_calculatePrice` uses raw EVM division in which division by zero returns 0 without reverting. Therefore, `SELL` orders with `makerAmount == 0` will produce as ask price of 0.00. `CalculatorHelper._isCrossing` then checks complementary orders as `buyPrice >= sellPrice`, so once a malformed `SELL` is interpreted as `sellPrice = 0`, any `BUY` with non-negative price passes as crossing, producing a false positive match condition.

Proof of Concept: The following test demonstrates the false crossing:

```
function test_PoC_ZeroMakerSellFalseCross() public pure {
    // SELL price = taker / maker, so maker=0 => div-by-zero => 0
    uint256 sellPrice = CalculatorHelper._calculatePrice(0, 1e18, Side.SELL);
    uint256 buyPrice = CalculatorHelper._calculatePrice(1e18, 1e18, Side.BUY);

    assertEq(sellPrice, 0, "expected zero ask due to EVM div-by-zero behavior");

    // Complementary crossing becomes: buyPrice >= sellPrice => 1e18 >= 0 (true)
    bool crossing = CalculatorHelper._isCrossing(sellPrice, buyPrice, Side.SELL,
    ↪  Side.BUY);
    assertTrue(crossing, "expected false crossing");
}
```

Recommendation: Invalid quotes arising from zero sell quantity should be explicitly prevented from creating a false crossing as the sell side has no meaningful price. Such degenerate `SELL` orders with `makerAmount == 0` should be rejected instead of being matched. Alternatively, consider returning false (or revert) when a `SELL` denominator would be zero.

Polymarket: Fixed in PR 68.

Cantina Managed: Verified. The dead code has been removed.

3.3.9 Users should be paused within `blockPausedAt`

Severity: Informational

Context: UserPausable.sol#L20, UserPausable.sol#L25

Description: `UserPausable::isUserPaused` currently checks whether the block number strictly exceeds the timestamp at which the user is considered paused. This corresponds to the addition of the `userPauseBlockInterval` to the current block, so it may instead be preferable to have the pause take effect within this same block rather than after it.

Recommendation: Consider changing the condition to `block.number >= blockPausedAt`.

Polymarket: Fixed in PR 70.

Cantina Managed: Verified. The pause now takes effect within the interval block.

3.3.10 Repeated `pauseUser()` executions will delay pausing

Severity: Informational

Context: UserPausable.sol#L24-L26

Description: `UserPauasable::pauseUser` does not prevent repeated pause execution which will push back the effective block in which the account is considered paused.

Recommendation: Consider allowing new pause executions only if the user is not already paused.

Polymarket: Fixed in PR 67.

Cantina Managed: Verified. Users can now only pause when not already paused.

3.3.11 User accounts can be unpaused before or after the effective pause block has passed

Severity: Informational

Context: UserPausable.sol#L31-L32

Description: `UserPausable::unpauseUser` can currently be called at any point before or after the effective pause block has passed.

Recommendation: Consider restricting this function to execute only when the user is already paused, or document this behavior if intended.

Polymarket: Acknowledged. Users can cancel pending pause by design, so this is working as intended.

Cantina Managed: Acknowledged.

3.3.12 Incorrect `takerFillAmount` documentation in `CTFExchange::matchOrders`

Severity: Informational

Context: CTFExchange.sol#L37-L38

Description: The `CTFExchange::matchOrders` NatSpec appears to have been miscopied for `takerFillAmount` which is always in terms of the maker amount units.

Recommendation: Remove the trailing reference to "taker amount".

Polymarket: Fixed in PR 70.

Cantina Managed: Verified.

3.3.13 Invalidated pre-approved orders can be reapproved without restriction

Severity: Informational

Context: Signatures.sol#L23-L29

Description: Pre-approved orders can be invalidated by the operator via `CTFExchange::invalidatePreapprovedOrder`; however, such orders can be pre-approved again with no restriction after invalidation.

Recommendation: Consider preventing invalidated orders from being pre-approved.

Polymarket: Acknowledged. Both functions are operator-only. `preapproveOrder()` and `invalidatePreapprovedOrder()` are gated by `onlyOperator()`. The operator is already trusted to control the entire matching flow.

Cantina Managed: Acknowledged.

3.3.14 Redundant non-zero fee validation before `_chargeFee()` calls

Severity: Informational

Context: Trading.sol#L201, Trading.sol#L231, Trading.sol#L290-L292, Trading.sol#L417-L419, Trading.sol#L552-L554

Description: `Trading::_chargeFee` validates that the fee to be charges is greater than zero. Therefore, in all instances of `_chargeFee()` invocation, it is not necessary to include this repeated validation.

Recommendation: Remove the redundant non-zero fee validation.

Polymarket: Fixed in PR 68.

Cantina Managed: Verified. The repeated validation has been removed.

3.3.15 Missing ci foundry profile leaves ffi enabled in `pull_request` tests

Severity: Informational

Context: foundry.toml#L1-L3, test.yml#L1-L11

Description: The CI workflow defined in `.github/workflows/test.yml` sets `FOUNDRY_PROFILE=ci`, but `foundry.toml` does not define `[profile.ci]`. In practice, `forge test` uses the default profile, which has `ffi = true`, so untrusted PR test code can invoke `vm.ffi(...)` and run host commands on the GitHub runner. In a more complex CI workflow, or if using self-hosted runners, a malicious PR could attempt exfiltration of runner-local context,e.g. workspace files, environment variables, etc.

Recommendation: Consider adding an explicit CI profile in which `ffi = false`.

Polymarket: Fixed in PR 70.

Cantina Managed: Verified. An explicit profile has been added with ffi disabled.

3.3.16 Auth mappings should use boolean as flags instead of `uint256`

Severity: Informational

Context: Auth.sol#L10

Description: The `admins` and `operators` mappings in `Auth` map from `address` to `uint256` which is used as an indicator of whether an address has the admin or operator role. The semantic type is boolean and using that type would be clearer and more idiomatic as well as reduce the ability of a developer misusing the wrong uint256 for the role they wish to flag.

Recommendation: Consider using `mapping(address => bool)` instead for `admins` and `operators`.

Polymarket: Fixed in PR 70.

Cantina Managed: Verified. The mappings now use booleans.

3.3.17 Missing NatSpec on public/external functions

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: Several public/external functions have no NatSpec at all.

Contract	Functions
`CtfCollateralAdapter`	`splitPosition`,`mergePositions`, and `redeemPositions`
`NegRiskCtfCollateralAdapter`	`convertPositions`
`Assets`	`getCollateral`, `getCtf`, and `getOutcomeTokenFactory`
`Auth`	`isAdmin` and `isOperator`

Recommendation: Add NatSpec for future readers and developers to understand exactly how you intend this code to work.

Polymarket: Fixed in PR 79.

Cantina Managed: Verified.

3.3.18 Internal/Private Functions Missing Leading Underscores

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: There are several internal or private functions that do not cohere to the codebase style of naming these functions with leading underscores.

Contract	Functions

`Signatures`	`isPreapproved`
`Fees`	`validateFeeWithMaxFeeRate`
`CalculatorHelper.sol`	`calculateTakingAmount`, `isCrossing`
`PolySafeLib`	`computeBytecodeHash`
`PolyProxyLib`	`getProxyWalletAddress`
`Create2Lib`	`computeCreate2Address`
`CTFHelpers`	`positionIds`, `partition`

Recommendation: Consider maintaining a consistent style across the codebase with internal and private function names.

Polymarket: Fixed in PR 75.

Cantina Managed: Verified.